实验报告：两层ReLU神经网络模拟函数练习

代码详见 `fnn-numpy-function-simulation.ipynb` 文件，已写出详细的注释并为代码按照语义分好了段，方便理解我的思路，下面是报告内容：

---

1. 函数定义

本实验的目标是使用一个两层前馈神经网络，模拟以下非线性函数：

$f(x) = 0.5\cos(2x) + 0.25\sin(3x)$ ，定义域为 $(x \in [-\pi, \pi])$

该函数包含余弦和正弦成分，具有一定的波动性，适合检验神经网络的拟合能力。

2. 数据采集

- **训练集**：在定义域内等间距采样 1000 个点，构成训练数据。每个点形状为 `(x, f(x))` ， `x` 为输入， `f(x)` 为标签。
- **测试集**：从同一区间内随机采样 300 个点，用于评估模型在未见数据上的表现。

数据均以 `float32` 类型存储，确保与 NumPy 计算兼容。

3. 模型描述

网络结构

- **输入层**：原始输入为 1 维（x），为引入偏置项，在输入后拼接一列 1，实际输入维度为 2。
- **隐藏层**：包含 400 个神经元，使用 ReLU 激活函数。
- **输出层**：线性层，1 个神经元，输出预测值。

参数初始化

- 权重矩阵 `W1` (2×400) 和 `W2` (400×1) 初始化为小随机数（均值为 0，标准差 0.01），以避免梯度消失或爆炸。
- 输出层偏置 `b2` 初始化为 0。

前向传播与反向传播

- 手动实现了 `Matmul` 和 `Relu` 类，分别负责矩阵乘法和 ReLU 激活的前向与梯度计算。
- 损失函数采用均方误差（MSE）。
- 反向传播中，损失对输出的梯度为 $(\frac{2}{N}(\hat{y} - y))$

- 然后通过链式法则逐层计算参数梯度。
- 参数更新使用梯度下降法，学习率固定为 `lr = 0.005`。

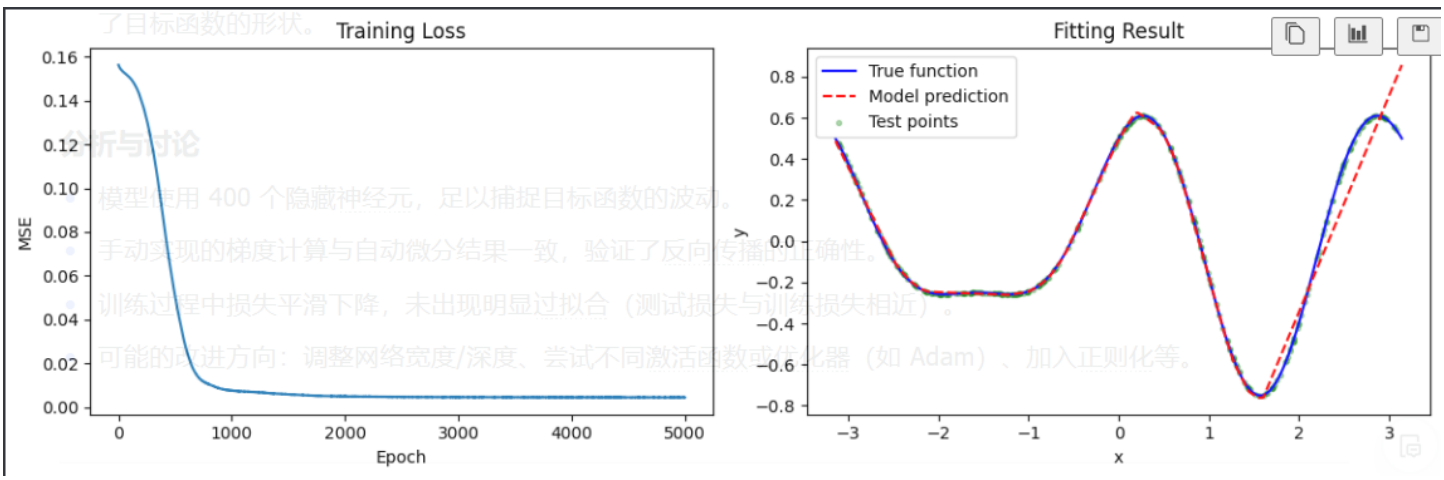
训练细节

- 训练轮数：5000 个 epoch。
- 批大小：32。
- 每个 epoch 前对训练数据随机打乱，以增强泛化能力。

4. 拟合效果

训练过程中，损失函数稳步下降，最终训练损失约为 **0.00436**，测试损失为 **0.00414**，表明模型在训练集和测试集上均表现良好。

下图为训练损失曲线和模型拟合效果的可视化：



- **左图**：训练损失随 epoch 下降，前 1500 步快速下降，之后趋于平稳。
- **右图**：蓝色曲线为真实函数，红色虚线为模型预测，绿色点为测试样本。两者几乎重合，说明网络成功学习到了目标函数的形状。

分析与讨论

- 模型使用 400 个隐藏神经元，足以捕捉目标函数的波动。
- 手动实现的梯度计算与自动微分结果一致，验证了反向传播的正确性。
- 训练过程中损失平滑下降，未出现明显过拟合（测试损失与训练损失相近）。
- 可能的改进方向：调整网络宽度/深度、尝试不同激活函数或优化器（如 Adam）、加入正则化等。